2025 – 2026

COMP7705 Capstone Project

OptiTrade Copilot

An AI-Driven Trading Portal with Interactive Dynamic Canvas

Project Proposal

Student Name	Student ID
Wong Wai Wing	3036383513
Ng Wing Yin	3036383927
Hui Nga Chi	3036198774
Ng Sui Yat	3036383549
Cheung Ching Nam	3036382959

Abstract

Professional traders operate in a fragmented information environment where portfolio data, real-time market prices, technical indicators, and financial news are spread across separate applications. This fragmentation increases cognitive load, slows decision-making, and makes it difficult to verify the basis for trading decisions. Although large language model (LLM) assistants can summarize and explain market conditions, their outputs often lack grounding in the exact data a trader is viewing—creating material risk from hallucinations and insufficient provenance. Existing alerting systems are typically generic and difficult to personalize, leading to missed events or excessive low-value notifications.

This project proposes OptiTrade Copilot: an integrated, customizable financial dashboard combining three core capabilities:

Dynamic Widget Canvas	A drag-and-drop workspace for charts, portfolio analytics, and AI-enhanced news — allowing each user to configure a personalized view of strategy-relevant signals.
Context-Aware AI Chat Panel	A conversational AI interface whose responses are explicitly grounded in timestamped 'context cards' exported from on-screen widgets, ensuring every answer is traceable to real data.
Real-Time Alerts & Monitoring	Near-real-time notifications driven by user-defined rules tied to technical indicators, portfolio thresholds, and news risk signals — delivering position-aware, high-precision alerts.

The system targets three measurable outcomes: reduced decision latency through personalized views, improved trust via verifiable AI snapshots, and higher signal-to-noise in alerts via position-aware personalization. An MVP will demonstrate end-to-end functionality across widget customization, grounded conversational support, and at least one operational notification workflow.

1 Background

Despite the proliferation of analytics platforms and increasingly capable LLMs, professional traders continue to operate in a fragmented, noisy, and difficult-to-verify information environment. Portfolio positions, real-time market data, technical charts, and news feeds commonly reside in separate applications; integrating those sources to form a coherent, auditable basis for decisions remains a largely manual task. At the same time, conversational agents built on LLMs can provide fluent explanations and summaries, but their outputs frequently lack a verifiable link to the specific data a trader sees. This combination — disjointed interfaces plus unverifiable model output — creates three concrete problems that degrade decision quality, increase operational risk, and impede auditability [7][8][9].

Problem Overview

#	Problem	Root Cause	Impact
P1	Non-personalized dashboards cause information overload	Generic views designed for multiple user types display irrelevant signals	Slower event recognition; higher error rate for time-sensitive decisions
P2	LLM hallucinations in conversational interfaces	Models lack grounded access to the trader's exact data snapshot	Unverifiable claims; compliance and regulatory exposure
P3	Limited personalized monitoring and notification capability	Coarse, pre-configured alerts not bound to user holdings or strategy	Missed opportunities or alert fatigue from low-value notifications

1.1 Problem 1 — Non-Personalized Dashboards and Information Overload

Empirical work in cognitive load theory and human-computer interaction shows that decision performance degrades as the quantity and heterogeneity of displayed information increase, unless information is organised to match the user's goals and mental model [2][3]. In trading contexts, generic dashboards present a broad set of metrics aimed at many user types simultaneously. The consequence is twofold: experienced traders must expend attention to filter out irrelevant signals; less experienced users face steep learning costs and are more prone to misinterpretation. Both outcomes increase the latency of recognising high-priority events and elevate the likelihood of erroneous actions. A targeted, user-centric view — one that prioritizes the instruments, risk metrics, and alerts aligned with a trader's positions and strategy — reduces extraneous cognitive load and enables faster, more accurate decisions [1][4].

1.2 Problem 2 — Hallucination and Insufficient Context in Conversational Interfaces

LLMs are susceptible to confidently stated inaccuracies (hallucinations) when the underlying model has incomplete or no access to the factual state it is asked to reason about [5][6][9]. In a trading setting this risk becomes material: a trader asking "Which position has the largest intraday loss?" expects an answer traceable to the portfolio and market data visible at that exact moment. If the assistant produces an answer lacking provenance — no link to holdings, quote timestamps, or chart view — the reply cannot be audited or validated, and trust degrades. From a compliance perspective, unverifiable advice carries additional regulatory and legal exposure. To be practically useful, conversational interfaces must (a) receive and retain precise, timestamped snapshots of the user's portfolio and screen state, and (b) constrain model outputs to cite snapshot elements or explicitly label statements as inference beyond the snapshot [7][8].

Recent work on RAG-augmented financial systems demonstrates that grounding LLM responses in verifiable data significantly reduces hallucination rates [8][9]. QuantMCP [9] and FinBloom [10] both confirm that retrieval-augmented architectures substantially improve factual accuracy in real-time financial analysis tasks.

1.3 Problem 3 — Limited Capability for Personalized Monitoring and Notifications

Monitoring systems in many trading platforms provide only coarse-grained, pre-configured alerts (price crossing a threshold, simple indicator triggers) that are not tightly coupled to a user's actual holdings, risk tolerance, or bespoke rules [2][3]. Creating more expressive, personalized monitors typically requires coding skills or stitching together multiple services, which raises the setup cost and reduces adoption. The result is that traders miss opportunities for timely intervention or receive excessive low-value alerts. A practical solution must enable users to define and persist monitoring rules directly bound to their positions and strategy via an intuitive interface — guided templates or natural-language rule authoring — that compiles into verifiable, executable conditions and routes notifications through preferred channels [11].

1.4 Synthesis: Integrated, Explainable, and Verifiable Decision Support

A viable architectural response synthesises three capabilities: (1) tight integration of real-time market and portfolio data so the system maintains a machine-readable, timestamped representation of the user's state; (2) explainable analytics producing concise, evidence-linked observations rather than opaque recommendations; and (3) context grounding for conversational agents — exportable snapshots of the exact screen and portfolio state that accompany any AI-generated statement. Combined, these capabilities reduce context switching, lower the cognitive burden of filtering irrelevant signals, and materially improve the traceability and auditability of AI-assisted reasoning [5][6][7].

1.5 Measurable Objectives and Expected Benefits

Objective	Description	Measurement Method	References
Reduce decision latency	Personalized, strategy-relevant views surface prioritized events faster	Task completion times in controlled user studies	[1][2]
Higher alert signal-to-noise	Position-aware, user-customizable monitors reduce irrelevant notifications	Alert precision/recall against an event corpus or user feedback	[2][3][11]
Reduce hallucination rate	Constrain model outputs to cited snapshot contents and provenance markers	Hallucination rate on benchmarked question sets	[6][8][9]

2 Literature Review

2.1 Existing Financial Dashboard Platforms

Financial dashboard platforms have evolved from simple charting tools into powerful systems for real-time analysis, trading, and decision-making. The three most relevant reference platforms are compared below.

Platform	Key Strengths	Limitations Addressed by This Project
TradingView	Advanced multi-chart layouts, 100+ chart types, hundreds of built-in indicators, real-time screeners, social idea-sharing, broker integrations	No AI-grounded conversational layer; alerts are symbol-centric, not portfolio-aware; no context-card provenance mechanism
Interactive Brokers TWS	Professional execution, customizable Mosaic interface, global markets, 100+ order types, strong Python/Java APIs	Interface complexity creates high cognitive load; minimal natural-language support; no AI chat or sentiment analysis
Perplexity Finance	Conversational AI-first, 40+ live finance integrations, brokerage account connectivity via Plaid, AI-generated risk assessments	Context grounding is opaque (no exportable snapshot cards); limited widget personalisation; no rule-based alert authoring

2.2 AI Technologies for Financial Analysis

A growing body of literature demonstrates that AI and ML techniques can substantially enhance financial platforms across several dimensions.

AI/ML Category	Techniques	Relevance to OptiTrade Copilot
Deep Learning for Market Prediction	LSTMs, Transformers, reinforcement learning; handle non-stationary market data better than traditional methods	Underpins momentum snapshot and chart insight generation in the Chart Widget
NLP & Sentiment Analysis	BERT-based models (e.g., FinBERT), LLM-based sentiment scoring of financial news, earnings calls, and social media [12][13]	Powers per-article sentiment scores and risk tagging in the News Widget
LLM Grounding & Hallucination Mitigation	Retrieval-Augmented Generation (RAG), context cards, provenance markers [8][9][10]	Core mechanism for the Context-Aware AI Chat Panel
Rule-Based & Event-Driven Alerting	Threshold monitoring, Celery/Redis task queues, multi-channel notification pipelines [11]	Scheduler for the Notification & Alert system
Explainable AI (XAI)	Evidence-linked outputs, confidence labels, reasoning transparency [5]	Disclaimer and reasoning indicators in the AI Chat Panel

2.3 Gap Analysis and Project Positioning

The table above highlights a gap in existing platforms: none combines (a) a fully personalized widget canvas, (b) AI responses explicitly grounded in verifiable on-screen data snapshots, and (c) position-aware, rule-based

alert authoring in a single integrated system. OptiTrade Copilot is designed to close this gap, drawing on research in information overload reduction [1][2], RAG-based hallucination mitigation [8][9][10], and intelligent alert systems [11].

3 Methodology

3.1 Technical Architecture Overview

The system is structured around three loosely-coupled layers that communicate through well-defined APIs.

Layer	Components	Technology Stack (example, TBC)
Frontend – Analysis Dashboard	Grid-based widget canvas, Chart Widget, Portfolio Widget, News Widget, AI Chat Panel, Alert configuration UI	React, react-grid-layout, Recharts / Highcharts, Redux / Context API, Tailwind CSS, Shadcn UI
Backend – API & Data Service	REST / WebSocket endpoints for market data, portfolio, news, widget state, and alert rules; candle cache; broker mock	Next.js API routes, Python FastAPI (AI service), PostgreSQL / MongoDB, Redis, Celery, WebSocket or SSE
AI Platform	LLM inference, context engineering, momentum & sentiment analysis, RAG grounding engine	LLM Provider (e.g., Azure OpenAI Services), LangChain, custom context-card store,

Commented [d1]: WebSockets != SSE

A context management sub-system sits across all three layers: it stores widget-exported data objects, assigns each a unique contextId, and injects the relevant context blocks into LLM prompts. This is the primary mechanism through which the system achieves provenance and hallucination reduction [7][8].

3.2 Dynamic Customizable Widget System

The Dynamic Customizable Widget system introduces a flexible, adaptive canvas that allows users to organise and personalize their workspace. Users can add, remove, and rearrange widgets to create a modular dashboard tailored to individual workflows. Widgets are designed to work together; news articles can be sent to the AI Chat Panel for deeper analysis, while portfolio alerts can be triggered by insights from the Chart Widget.

3.2.1 Canvas and Layout

The canvas is built on a responsive, grid-based design ensuring widgets can be dragged, resized, and snapped into place. Users can save customised layouts, allowing them to switch between configurations suited to different workflows (e.g., a trading-focused setup vs. a research-oriented dashboard). The system supports dynamic real-time updates.

Grid framework	react-grid-layout with 12-column responsive grid; snap-to-grid with visual guides
State persistence	Widget configurations (type, position, size, settings) persisted to the database; restored on login
Multi-layout support	Users can save and switch between named layout profiles (e.g., 'Trading', 'Research')
Responsive design	Layouts adapt to desktop, tablet, and mobile screen sizes

3.2.2 Widget Catalogue

Each widget type brings specialized capabilities to the canvas. The four core widget types are summarised below.

Widget	Primary Function	AI Enhancement	Chat Context Export
Price & Technical Indicator Chart	Candlestick/line charts with RSI, MACD, Bollinger Bands, MA, volume for up to 5 symbols	Momentum Snapshot (Overbought / Oversold / Neutral) with 1–3 bullet rationale	Symbol, timeframe, interval, indicator params, up to 200 candles, derived statistics
Portfolio Full (Large)	Holdings table, donut allocation chart, P/L metrics, concentration risk analysis	AI health summary: diversification assessment, top contributors, risk concentration	Total portfolio value, individual holdings, computed P/L fields
Portfolio Tiles (1×1)	P/L Tile: total day P/L; Top Mover Tile: largest position mover today	N/A (display-only tile)	Single metric the tile displays
News Widget	Filtered market news feed by watchlist, portfolio, or region with full-text article view	2–5 bullet summary, sentiment score [−1, +1], risk/event tags per article	Article IDs, titles, sources, timestamps, AI summaries, sentiment scores, risk tags

3.2.3 User Interface and Experience

The system's UI prioritizes simplicity and usability. Users add widgets via a library interface and drag-and-drop them onto the canvas. Each widget includes a settings menu for quick configuration of data sources, refresh intervals, and alert thresholds. The interface adheres to a consistent design language ensuring a professional appearance across all widgets, while responsive design makes the system accessible on desktops, tablets, and mobile devices. The canvas state is automatically saved.

3.2.4 Price and Technical Indicator Chart Widget — Detail

The Chart Widget is a central feature designed for interactive, customizable stock price visualisation and technical analysis.

Controls and Chart Types

Symbol support	Up to 5 symbols, each in its own sub-tab
Timeframes	1D, 5D, 1M, 3M, 6M, 1Y, 5Y, MAX
Intervals	1m, 5m, 15m, 1h, 1d (constrained by timeframe)
Chart types	Candlestick (OHLC detail) and Line (trend overview)
Interaction	Pan, zoom, crosshair with OHLC tooltip, reset-view button
Price markers	Current price marker; absolute and percentage day change displayed

Technical Indicators

Indicator	Default Config	Display Location	Key Signals
Moving Average (MA)	20-period; switchable to 50 or 200	Overlay on price chart	Trend direction, crossovers

Relative Strength Index (RSI)	14-period	Separate sub-panel	Overbought (>70), Oversold (<30)
MACD	12/26/9 standard settings	Separate sub-panel	Signal line crossover, histogram divergence
Bollinger Bands	20-period MA, ±2 std dev	Overlay on price chart	Volatility, breakout signals
Volume	Bar chart, colour-coded	Below price chart	Green = up-day, Red = down-day

AI-Powered Momentum Snapshot

The Momentum Snapshot feature analyses chart data and labels the stock as Overbought, Oversold, or Neutral based on RSI values, accompanied by a 1–3 bullet-point rationale. All outputs are accompanied by the disclaimer: "This is not financial advice." Users may also request a full textual chart explanation covering trend direction, support/resistance levels, volatility insights, and momentum state.

Acceptance Criteria

Indicator accuracy	MA, RSI, MACD, and Bollinger Bands match expected values for a given OHLCV dataset
Data alignment	Candlestick data aligns with selected timeframe and interval
Interactive features	Pan, zoom, reset, and crosshair/tooltip function without errors
AI Momentum Snapshot	Labels and rationale are generated within 3 seconds; disclaimer is always present
Chat context export	Exported payload correctly captures symbol, timeframe, indicator params, and candle summary

3.2.5 Portfolio Widget — Detail

The Portfolio Widgets provide an intuitive interface for managing and analysing investment holdings. Three variants address different viewing needs.

Variant	Size	Features
Portfolio Full	Large (configurable)	Holdings table, donut allocation chart, total value, day P/L, cost basis, unrealized P/L, AI health insights, connect/disconnect/sync/refresh broker controls
Portfolio P/L Tile	1×1	Displays total day P/L; Add to Chat Context button only
Top Mover Tile	1×1	Displays the largest position mover today; Add to Chat Context button only

The Portfolio Full widget features a flexible header that adapts dynamically based on broker connection status. When disconnected, users can use a Mock Broker to simulate broker integration. Upon connection, Disconnect, Sync, and Refresh Quotes options become available. The AI health insight covers diversification assessment, top contributors, and concentration risk.

3.2.6 News Widget — Detail

The News Widget aggregates market news and uses AI to reduce reading time by summarising key insights and highlighting risks or sentiment. It focuses on providing concise, actionable information to help users quickly identify important market developments.

Feed view	List or card format with title, source, time, and snippet per article
Filters	By watchlist, portfolio holdings, or region (e.g., US, HK)
Per-article AI output	2–5 bullet summary; sentiment score [−1, +1] with colour cue; risk/event tags with rationale tooltip and High/Low level
Refresh	Scheduled (default every 15 minutes) and manual refresh
Chat context export	Article IDs, titles, sources, timestamps, AI summaries, sentiment scores, risk tags, and a grounding note

3.3 Context-Aware AI Chat Panel

The Context-Aware AI Chat Panel provides users with an intuitive and interactive way to engage with AI-driven insights while maintaining a clear connection to their current activities and data. By enabling context-aware conversations, this panel ensures responses are grounded in the user's specific widgets and data [8][9].

3.3.1 UI/UX

Layout	Collapsible right sidebar or modal overlay providing full-height chat experience
Streaming responses	Real-time delivery via SSE or WebSocket; response streams as it is generated
Context Cards Bar	Located at the top of the chat panel, displays active contexts (e.g., [AAPL Chart], [Portfolio], [Tech News])
Input box	Supports Markdown formatting including tables, bullet points, and code blocks
Action buttons	Clear Context (removes all contexts and resets conversation); optional Show Reasoning/Status indicator

3.3.2 Core Behaviours

Context push & storage	Widgets push context blocks (data from charts, portfolios, or articles) to the backend; each block is stored and assigned a unique contextId
Grounded responses	The LLM is instructed to reference attached contextIds and only make claims supported by the snapshot; unsupported claims are labelled as inference
Safety constraints	The system avoids generating outputs unsupported by provided context; disclaimers are included as appropriate
Multi-context queries	Users can attach multiple context cards (e.g., chart + portfolio + news) in a single conversation turn

3.3.3 Value and Differentiation

The context-card provenance mechanism is the key differentiator from general-purpose LLM interfaces. By tying each AI response to a concrete, timestamped data snapshot, OptiTrade Copilot addresses the hallucination and auditability problems identified in Problem 2. This directly operationalizes recommendations from the RAG hallucination mitigation literature [8][9][10], which demonstrates that verifiable evidence grounding is among the most effective techniques for reducing unsupported LLM claims in financial settings.

3.3.4 Acceptance Criteria — MVP

Streaming responses	Delivered seamlessly via SSE/WebSocket with no perceptible gaps or failures
Context cards bar	Accurately reflects all attached widget contexts; cards can be individually removed
Grounded answers	Responses cite the attached context (e.g., RSI value from the attached AAPL chart); unsupported claims are labelled
Multi-widget context	At least two widget types can be simultaneously attached and referenced in a single response

3.4 Real-Time Monitoring, Notifications, and Alerts

The Notifications and Alerts feature keeps users informed about important updates through scheduled email digests or rule-based alerts, ensuring proactive awareness of key developments without constant platform monitoring.

3.4.1 Alert Rule Types

Source Widget	Trigger Type	Example Condition	Priority
Chart Widget	Technical indicator threshold	RSI > 70 or RSI < 30; price crosses a specified level; 50 MA crosses above 200 MA	High (MVP)
Portfolio Widget	Portfolio metric threshold	Day P/L drops by more than 5%; top mover exceeds ±3% in a single session	High (MVP)
News Widget	Sentiment / risk event	Article with risk level = High for a portfolio holding; extreme sentiment score	Optional (post-MVP)

3.4.2 Technical Implementation

Task queue	Celery with Redis or RabbitMQ; workers periodically fetch market data and evaluate user-defined rules
Rule evaluation	Rules compiled into parameterised condition functions; evaluated against latest market/portfolio snapshots
Notification delivery	Email via SendGrid or AWS SES; in-dashboard real-time alerts via WebSocket
Retry & reliability	Exponential back-off on delivery failures; detailed logging; dead-letter queue for undeliverable messages
Alert management UI	Rules listed per widget with status (active / paused); configurable trigger conditions, frequency, and delivery method

3.4.3 Alert Configuration UI

The notification system is accessible within each widget's settings via a Create Alert entry point. A configuration modal allows users to specify: trigger conditions (e.g., RSI > 70, price crosses $150), notification

frequency (real-time or scheduled digest), and delivery method (email or in-dashboard). All saved alert configurations are visible within the widget settings.

3.4.4 Acceptance Criteria

End-to-end path	At least one notification workflow functions end-to-end: create rule → execute scheduled run → deliver and log email notifications
Alert persistence	Saved rules survive page reload and session restart
UI completeness	Rules for trigger-based alerts not yet implemented are labelled 'Coming soon' and remain visible
Widget canvas	All widgets can be added/removed without blank states
Add to Chat Context	Every widget supports the Add to Chat Context action; chat displays corresponding active context chips
Error handling	If a data provider is rate-limited or down, widget displays clear error message with retry option

4 Project Schedule

The project is divided into four phases, each building on the previous to deliver a complete and tested system.

Phase	Period	Milestones	Est. Hours	Deliverables
Phase 0: Proposal & Setup	Mar 2026 (by 20 Mar)	Finalize and submit Detailed Proposal	10	Detailed Project Proposal Project Webpage (initial)
		Confirm mentor and sub-deadlines	10	
		Set up monorepo: Next.js frontend + Python AI service	10	
		Configure Convex, Clerk auth, Zod validation	10	
		Establish shared component library (Shadcn + Tailwind)	10	
		Define API contract stubs for all widget endpoints	10	
		Set up CI/CD pipeline + linting/testing framework	10	
		Design and provision database schema	10	
		Stand up project webpage	10	
Phase 1: Core Infrastructure & MVP Widgets	Mar – Apr 2026 (by 14 Apr)	Broker Mock endpoints	15	Progress Update 1
		Quote fetching & derived calculations	15	
		Top Mover algorithm	10	
		Market Data service & candle cache	15	
		Dashboard layout & widget framework	20	
		Edit mode: drag/drop, resize, reorder	15	
		Portfolio Full widget	20	
		P/L Tile & Top Mover Tile	10	
		Price Chart (TradingView) integration	20	
		Candlestick/Line chart + timeframe/interval selectors	15	
Phase 2: AI Features & News Widget	Apr – May 2026 (by 5 May)	AI Insights for Portfolio Full	15	Progress Update 2
		AI Momentum Snapshot for Chart Widget	15	
		News feed aggregation + filtering	15	
		Per-article AI Insight (summary, sentiment, risk tags)	20	
		Context-Aware AI Chat Panel — UI + streaming	20	
		Context management system (contextId store)	15	
		Chat Context Export for all widgets	15	
		Grounded response system + safety constraints	15	
Phase 3: Alerts, Testing & Final Submission	May – Jun 2026 (by submission)	Alert rule authoring UI	15	Final Report Project Demo Project Webpage (final)
		Celery/Redis scheduler implementation	20	
		Email notification delivery (SendGrid)	15	

		In-dashboard real-time alert integration	15	
		End-to-end testing across all features	20	
		Performance optimization and load testing	10	
		Final documentation and project webpage update	10	
		Presentation preparation	10	

References

The following references are cited in the proposal using numbered inline markers.

#	Authors / Year	Citation
[1]	Ramdani et al. (2025)	Dynamic Dashboard Optimization with AI: Enhancing User Experience and Improving Decision Making Process. IEEE ICOA 2025. DOI: 10.1109/ICOA66896.2025.11236939
[2]	Bernales, Valenzuela & Zer (2023)	Effects of Information Overload on Financial Markets: How Much Is Too Much? Federal Reserve International Finance Discussion Papers, No. 1372. DOI: 10.17016/ifdp.2023.1372
[3]	Zafar et al. (2025)	From Digital Tools to Decision Outcomes: Understanding Investment Quality in FinTech Environments. Academic Journal, DOI: 10.63056/acad.004.04.1457
[4]	Myllynen & Kamau (2025)	The Evolution of Dashboard Design: Best Practices for Data Visualization in Decision Support Systems. Computational Engineering Journal. DOI: 10.54660/.ijeca.2025.1.6.01-17
[5]	Doshi-Velez & Kim (2017)	Towards a Rigorous Science of Interpretable Machine Learning. arXiv:1702.08608
[6]	Kang & Liu (2023)	Deficiency of Large Language Models in Finance: An Empirical Examination of Hallucination. arXiv:2311.15548
[7]	Hiriyanna & Zhao (2025)	Multi-Layered Framework for LLM Hallucination Mitigation in High-Stakes Applications. Computers, MDPI. DOI: 10.3390/computers14080332
[8]	Wang & Li (2025)	Artificial Intelligence for Quantitative Finance: A RAG-Augmented Multi-Agent Framework. ACM. DOI: 10.1145/3788149.3788249
[9]	Zeng (2025)	QuantMCP: Grounding Large Language Models in Verifiable Financial Reality. arXiv:2506.06622
[10]	Sinha, Agarwal & Malo (2025)	FinBloom: Knowledge Grounding Large Language Model with Real-time Financial Data. Knowledge-Based Systems. DOI: 10.1016/j.knosys.2026.115559
[11]	Li & Huang (2026)	Real-time Multithreaded Stock Monitoring and Alert System with Enterprise WeChat Notification and Threshold Prediction. SPIE. DOI: 10.1117/12.3104110
[12]	Jiang & Zeng (2025)	Financial Sentiment Analysis Using FinBERT with Application in Predicting Stock Movement. arXiv:2306.02136
[13]	Kirtac & Germano (2024)	Sentiment Trading with Large Language Models. Finance Research Letters. DOI: 10.1016/j.frl.2024.105227